

JavaScript Type Coercion and Comparisons

~ Vijay Kumar

Preface

JavaScript is one of the most widely used languages in the world, powering everything from dynamic websites to server-side applications. While its versatility and flexibility make it powerful, one of the most confusing parts of JavaScript is its **type coercion** and comparison behavior. Many developers, even experienced ones, often stumble on tricky edge cases when comparing values.

This report is written for developers who are familiar with JavaScript but want to gain a **deeper understanding** of how comparisons work under the hood. By mastering these concepts, you'll eliminate confusion, debug faster, and write more predictable code.

Table of Contents

1. Preface
 2. Introduction to Type Coercion
 3. Equality and Strict Equality (`==` vs `===`)
 4. Boolean Comparisons
 5. String Comparisons
 6. Number Comparisons
 7. Null and Undefined
 8. Objects in Comparisons
 9. NaN Rules
 10. Relational Comparisons (`<`, `<=`, `>`, `>=`)
 11. Conversion Reference Tables
 12. Definitions & Rules Recap
 13. Common Gotchas and Edge Cases
 14. Best Practices for Developers
 15. Key Takeaways
 16. Sources & Bibliography
 17. Interactive AI Practice Prompt
-

1. Introduction to Type Coercion

Type coercion is JavaScript's process of automatically converting one data type into another during evaluation. This implicit conversion is what often leads to unexpected results in comparisons.

There are **three main kinds of type coercion**:

- **To String**
- **To Number**
- **To Boolean**

For example:

```
String(123)    // "123"  
Number("456") // 456  
Boolean(0)     // false
```

When performing comparisons, JavaScript applies these coercions according to specific rules, which we will now break down in detail.

2. Equality (==) vs Strict Equality (===)

Definitions & Rules

- `==` checks **value equality with type coercion**.
- `===` checks **value and type equality without coercion**.

Examples:

```
5 == '5'    // true (string coerced to number)  
5 === '5'   // false (different types)  
0 == false  // true (false → 0)  
0 === false // false
```

Rule of thumb: Prefer `===` unless you explicitly want type coercion.

3. Boolean Comparisons

Rules

1. Booleans are always coerced to **numbers** first.
2. `true` → 1
3. `false` → 0
4. The other operand is then coerced to a number if needed.

Examples:

```
true == '1'    // true (true → 1, '1' → 1)
false == []    // true (false → 0, [] → '' → 0)
true == 'true' // false ('true' → NaN)
```

4. String Comparisons

Rules

1. String vs String → compared **lexicographically**.

```
'2' < '12' // false ('2' > '1')
```

2. String vs Number/Boolean → string coerced into number.

```
'5' < 10    // true ('5' → 5)
'0' < true  // true ('0' → 0, true → 1)
```

3. String vs null → both become numbers.

```
'0' < null // false ('0' → 0, null → 0)
```

4. String vs undefined/NaN → always false.

```
'10' < NaN    // false
'5' < undefined // false
```

5. Number Comparisons

Rules

1. Number vs String → string → number.

```
10 == '10' // true
```

2. Number vs Boolean → boolean → number.

```
1 == true // true
0 == false // true
```

3. Number vs null → null → 0.

```
null < 1 // true (0 < 1)
```

4. Number vs undefined → always false (undefined → NaN).

```
undefined < 1 // false
```

5. Number vs NaN → always false.

```
5 == NaN // false
```

6. Null and Undefined

Rules

- `null == undefined` → true.
- `null` converts to `0` in numeric comparisons, but not equality with numbers.
- `undefined` converts to `NaN` in numeric comparisons, so comparisons are false.

Examples:

```
null == undefined // true
null == 0          // false
null < 1           // true (null → 0)
undefined == 0     // false
undefined < 1      // false
```

7. Objects in Comparisons

Rules

1. Objects are converted to **primitives**.
2. Try `valueOf()`.
3. If not primitive, use `toString()`.
4. Then compare the primitive as per type.

Examples:

```
[2] < 3           // true ([2] → '2' → 2)
{} == '[object Object]' // true
[1] == 1          // true
```

8. NaN Rules

Rules

- `NaN` is never equal to anything, not even itself.

Examples:

```
NaN == NaN // false
NaN < 5     // false
Number.isNaN(NaN) // true
```

9. Relational Comparisons (`<`, `<=`, `>`, `>=`)

Rules

1. If both are strings → compare lexicographically.
2. Otherwise, convert both to numbers.
3. `null` → 0, `undefined` → `NaN`.
4. Any comparison with `NaN` → false.

Examples:

```
'2' < '12' // false
'2' < 12    // true ('2' → 2)
null < 1    // true (null → 0)
undefined < 1 // false
```

10. Conversion Reference Tables

String Conversions

```
String(123)      // '123'  
String(true)     // 'true'  
String(null)     // 'null'  
String(undefined) // 'undefined'  
String([1,2,3])  // '1,2,3'  
String([])       // ''  
String([null])   // ''  
String([undefined]) // ''
```

Number Conversions

```
Number('123')    // 123  
Number(true)     // 1  
Number(false)    // 0  
Number(null)     // 0  
Number(undefined) // NaN  
Number('')       // 0  
Number('text')   // NaN  
Number([1,2,3])  // NaN  
Number([])       // 0  
Number([null])   // 0  
Number([undefined]) // 0
```

Boolean Conversions

```
Boolean(0)       // false  
Boolean(1)       // true  
Boolean('')      // false  
Boolean('text')  // true  
Boolean(null)    // false  
Boolean(undefined) // false  
Boolean([])      // true  
Boolean([0])     // true  
Boolean([null])  // true  
Boolean({})      // true
```

11. Definitions & Rules Recap

- **Booleans** → always convert to number (true → 1, false → 0).
 - **Strings** → convert to number if compared with non-strings.
 - **Objects** → convert to primitive via `valueOf` or `toString`.
 - **null** → equal only to undefined, numeric coercion → 0.
 - **undefined** → equal only to null, numeric coercion → NaN.
 - **NaN** → never equals anything.
 - **String vs String** → lexicographic.
-

12. Common Gotchas and Edge Cases

- `[] == 0` → true (`[]` → '' → 0).
 - `[null] == 0` → true (`[null]` → '' → 0).
 - `[undefined] == 0` → true (`[undefined]` → '' → 0).
 - `{ } + [ ]` vs `[ ] + { }` → unexpected string concatenations.
-

13. Best Practices

1. Always use `===` unless coercion is **intended**.
 2. Avoid equality checks with `null` and `undefined` unless explicit.
 3. Be careful comparing objects.
 4. Explicitly convert values before comparing.
-

14. Key Takeaways

- Booleans → numbers.
 - Strings → numbers (unless both are strings).
 - Objects → primitives → numbers if needed.
 - null → 0, undefined → NaN.
 - NaN never equals anything.
 - String vs String → lexicographic.
-

Sources & Bibliography

- [MDN Web Docs: Equality comparisons and sameness](#)
 - [ECMAScript Language Specification](#)
 - [JavaScript.info](#)
 - ChatGPT collaborative exploration
-

Interactive AI Practice Prompt

Here's a prompt you can use with any AI tool (like ChatGPT) to test your knowledge:

Prompt: You are an interviewer testing my knowledge of JavaScript type coercion and comparisons. Ask me **20 questions**, one at a time. Wait for my answer before moving forward. If my answer is correct, explain why it's correct, then move to the next question. If my answer is wrong, explain the correct answer in detail, then proceed to the next. After 20 questions, calculate my score (e.g., 18/20), give me the percentage, and provide motivational feedback based on my performance (e.g., excellent, good, needs improvement).
